

tests/test_all.py

328 lines

```
1 """
2 test_all.py
3 =====
4 Dependency-free test suite (plain asserts, no pytest required) covering the
5 simulator, the identified model and the controller.
6
7 Run directly: python tests/test_all.py
8 Also run as part of: python run_all.py
9
10 The most important test here is `test_controller_against_foreign_plant`, which
11 drives the controller against a completely different plant that merely
12 satisfies the `WellSimulator` Protocol. That is what makes the claim "an
13 official simulator can be swapped in with zero controller changes" verifiable
14 rather than just asserted in a README.
15 """
16
17 from __future__ import annotations
18
19 import sys
20 import traceback
21 from pathlib import Path
22
23 ROOT = Path(__file__).resolve().parents[1]
24 sys.path.insert(0, str(ROOT / "src"))
25
26 import numpy as np
27
28 from config import CONTROLLER, DATA_DIR, LIMITS, PLANT, TS_HOURS
29 from controller import ChokeMPC
30 from model import OUTPUTS, WellModel
31 from simulator_interface import WellSimulator as WellSimulatorProtocol
32 from well_simulator import WellSimulator
33
34 FAILURES = []
35
36
37 def check(cond, msg):
38     if not cond:
39         raise AssertionError(msg)
40
41
42 # ----- simulator
43 def test_simulator_satisfies_protocol():
44     sim = WellSimulator()
45     check(isinstance(sim, WellSimulatorProtocol),
46            "WellSimulator does not satisfy the WellSimulator Protocol")
47
48
49 def test_reset_is_deterministic():
50     a = WellSimulator(seed=7)
51     b = WellSimulator(seed=7)
52     for u in [10, 30, 55, 20]:
53         for _ in range(3):
54             check(np.allclose(a.step(u), b.step(u)),
55                    "two simulators with the same seed diverged")
56
57     a.reset()
58     b.reset()
59     check(np.allclose(a.step(42), b.step(42)), "reset() did not restore the RNG")
60
61 def test_shut_in_gives_no_flow():
62     sim = WellSimulator(noise=False)
63     sim.reset()
64     for _ in range(10):
65         Q, WHP, FLP, BHP = sim.step(0.0)
66         check(abs(Q) < 1e-9, f"shut-in well still flowing at {Q}")
67         check(abs(BHP - PLANT.p_res) < 1e-6, "shut-in BHP should equal reservoir pressure")
```

```

68
69
70 def test_flow_increases_monotonically_with_choke():
71     sim = WellSimulator(noise=False)
72     qs = [sim.steady_state(u)["Q"] for u in range(0, 101, 5)]
73     for a, b in zip(qs, qs[1:]):
74         check(b >= a - 1e-9, "steady-state Q is not monotonically increasing in u")
75
76
77 def test_pressures_move_the_right_way():
78     """Opening the choke must raise Q and FLP, and lower WHP and BHP."""
79     sim = WellSimulator(noise=False)
80     lo, hi = sim.steady_state(30.0), sim.steady_state(60.0)
81     check(hi["Q"] > lo["Q"], "opening the choke did not increase Q")
82     check(hi["FLP"] > lo["FLP"], "opening the choke did not raise FLP")
83     check(hi["WHP"] < lo["WHP"], "opening the choke did not lower WHP")
84     check(hi["BHP"] < lo["BHP"], "opening the choke did not lower BHP")
85
86
87 def test_choke_input_is_clipped_and_finite():
88     sim = WellSimulator()
89     for u in [-50, -1, 0, 50, 100, 101, 1e6, float("inf")]:
90         out = sim.step(u)
91         check(all(np.isfinite(v) for v in out), f"non-finite output for u={u}")
92         check(out[0] >= 0.0, f"negative flow for u={u}")
93
94
95 def test_no_nan_under_long_random_drive():
96     rng = np.random.default_rng(0)
97     sim = WellSimulator(seed=3)
98     sim.reset()
99     for _ in range(500):
100         out = sim.step(rng.uniform(-10, 110))
101         check(all(np.isfinite(v) for v in out), "NaN/inf appeared during random drive")
102
103
104 def test_first_order_lag_is_present():
105     """A step change must NOT be reached instantly - there are real dynamics."""
106     sim = WellSimulator(noise=False)
107     sim.reset()
108     ss = sim.steady_state(50.0)
109     Q1, *_ = sim.step(50.0)
110     check(Q1 < 0.95 * ss["Q"], "output reached steady state in a single step (no lag)")
111
112
113 # ----- model
114 def test_model_roundtrips_through_disk():
115     m = WellModel.load(DATA_DIR / "fitted_model.json")
116     for k in OUTPUTS:
117         check(m.outputs[k].tau > 0, f"non-positive tau for {k}")
118         check(len(m.outputs[k].u_knots) == len(m.outputs[k].y_knots),
119              f"knot arrays disagree for {k}")
120
121
122 def test_model_prediction_shape_and_finiteness():
123     m = WellModel.load(DATA_DIR / "fitted_model.json")
124     y0 = {"Q": 100.0, "WHP": 500.0, "FLP": 220.0, "BHP": 2100.0}
125     traj = m.predict(y0, [40.0] * 8)
126     for k in OUTPUTS:
127         check(len(traj[k]) == 8, f"trajectory length wrong for {k}")
128         check(np.all(np.isfinite(traj[k])), f"non-finite prediction for {k}")
129         check(np.all(traj["Q"] >= 0.0), "model predicted negative flow")
130
131
132 def test_model_extrapolation_is_clamped():
133     m = WellModel.load(DATA_DIR / "fitted_model.json")
134     lo = float(m.outputs["Q"].y_ss(-40))
135     hi = float(m.outputs["Q"].y_ss(400))
136     check(np.isfinite(lo) and np.isfinite(hi), "extrapolation produced non-finite values")
137     check(m.outputs["Q"].is_extrapolating(400), "is_extrapolating failed to flag 400 %")

```

```

138
139
140 def test_model_tracks_plant_steady_state():
141     """The identified steady-state curve must match the plant closely - this is
142     what every constraint prediction rests on.
143
144     Accuracy is required where it changes decisions, not uniformly:
145
146     * u >= 55 % - BHP is below 1960 psi here and closing on its 1900 psi
147     limit, so this is the band where a prediction error can
148     actually cause a violation. Held to < 4 psi, well inside
149     the 15 psi safety margin.
150     * 35-95 % - normal operating band, BHP still 100+ psi clear.
151     * full range - the curve is very steep and strongly convex below ~30 %,
152     but BHP there is 500+ psi clear of any limit, so a looser
153     bound is the honest requirement.
154     """
155     m = WellModel.load(DATA_DIR / "fitted_model.json")
156     sim = WellSimulator(noise=False)
157
158     def worst_over(lo, hi):
159         return max(
160             abs(float(m.outputs["BHP"].y_ss(u)) - sim.steady_state(u)["BHP"])
161             for u in np.arange(lo, hi, 0.5)
162         )
163
164     constrained = worst_over(55, 95)
165     check(constrained < 4.0,
166           f"model/plant BHP error in the constrained band too large: {constrained:.1f} psi")
167     operating = worst_over(35, 95)
168     check(operating < 8.0,
169           f"model/plant BHP error in the operating band too large: {operating:.1f} psi")
170     overall = worst_over(15, 100)
171     check(overall < 20.0,
172           f"model/plant BHP error over the full range too large: {overall:.1f} psi")
173
174
175 # ----- controller
176 def _fresh_controller():
177     return ChokeMPC(model=WellModel.load(DATA_DIR / "fitted_model.json"))
178
179
180 def test_controller_respects_ramp_limit():
181     c = _fresh_controller()
182     meas = {"Q": 100.0, "WHP": 500.0, "FLP": 220.0, "BHP": 2100.0}
183     for u_now in [0.0, 12.5, 50.0, 97.0, 100.0]:
184         for target in [0.0, 50.0, 200.0, 1e4]:
185             d = c.step(meas, u_now, target)
186             check(abs(d.u_selected - u_now) <= CONTROLLER.max_move + 1e-9,
187                   f"ramp limit breached: {u_now} -> {d.u_selected}")
188             check(0.0 <= d.u_selected <= 100.0,
189                   f"choke out of range: {d.u_selected}")
190
191
192 def test_controller_never_selects_a_predicted_violation():
193     """If any candidate is feasible, the selected one must be feasible."""
194     c = _fresh_controller()
195     for bhp in [2900.0, 2200.0, 1960.0, 1930.0]:
196         meas = {"Q": 120.0, "WHP": 480.0, "FLP": 235.0, "BHP": bhp}
197         d = c.step(meas, 55.0, 250.0)
198         chosen = d.candidates[d.selected_index]
199         if any(x.feasible for x in d.candidates):
200             check(chosen.feasible,
201                   f"selected an infeasible candidate at BHP={bhp} despite safe options")
202
203
204 def test_controller_output_is_always_valid_even_when_all_infeasible():
205     """Deep inside a violation every candidate is unsafe; the controller must
206     still return a usable number and must not lurch."""
207     c = _fresh_controller()

```

```

208 meas = {"Q": 200.0, "WHP": 300.0, "FLP": 275.0, "BHP": 1500.0}
209 d = c.step(meas, 80.0, 250.0)
210 check(np.isfinite(d.u_selected), "fallback produced a non-finite choke position")
211 check(0.0 <= d.u_selected <= 100.0, "fallback produced an out-of-range choke position")
212 check(abs(d.u_selected - 80.0) <= CONTROLLER.max_move + 1e-9,
213 "fallback breached the ramp limit")
214
215
216 def test_controller_is_deterministic():
217     a, b = _fresh_controller(), _fresh_controller()
218     meas = {"Q": 90.0, "WHP": 600.0, "FLP": 215.0, "BHP": 2250.0}
219     for _ in range(5):
220         ua = a.step(meas, 40.0, 150.0).u_selected
221         ub = b.step(meas, 40.0, 150.0).u_selected
222         check(ua == ub, "controller is not deterministic")
223
224
225 def test_controller_holds_still_at_target():
226     """The deadband must stop the choke chattering once the target is met."""
227     model = WellModel.load(DATA_DIR / "fitted_model.json")
228     sim = WellSimulator(seed=11, noise=True)
229     sim.reset()
230     c = ChokeMPC(model=model)
231     u = 0.0
232     meas = {"Q": 0.0, "WHP": sim.history[-1]["WHP"],
233 "FLP": sim.history[-1]["FLP"], "BHP": sim.history[-1]["BHP"]}
234     us = []
235     for _ in range(60):
236         u = c.step(meas, u, 120.0).u_selected
237         Q, W, F, B = sim.step(u)
238         meas = {"Q": Q, "WHP": W, "FLP": F, "BHP": B}
239         us.append(u)
240     tail = np.array(us[-20:])
241     check(tail.std() < 0.25, f"choke still chattering at target: std={tail.std():.3f} %")
242     check(abs(np.mean(tail) - us[-1]) < 0.5, "choke not settled")
243
244
245 class _ForeignPlant:
246     """A deliberately DIFFERENT well that satisfies the same Protocol:
247     different pressures, different gains, different dynamics, different units
248     of nonlinearity. The controller has never seen it and has no access to
249     its internals - it only calls step()/reset()."""
250
251     def __init__(self):
252         self.reset()
253
254     def reset(self):
255         self.q, self.whp, self.flp, self.bhp = 0.0, 1200.0, 150.0, 2600.0
256         return self.q, self.whp, self.flp, self.bhp
257
258     def step(self, u):
259         u = float(np.clip(u, 0, 100))
260         q_ss = 190.0 * (u / 100.0) ** 0.75 # different characteristic
261         bhp_ss = 2600.0 - 5.2 * q_ss
262         whp_ss = bhp_ss - 1100.0 - 0.004 * q_ss ** 2
263         flp_ss = 150.0 + 0.55 * q_ss
264         a = 0.45 # different time constant
265         self.q += a * (q_ss - self.q)
266         self.whp += a * (whp_ss - self.whp)
267         self.flp += a * (flp_ss - self.flp)
268         self.bhp += a * (bhp_ss - self.bhp)
269         return self.q, self.whp, self.flp, self.bhp
270
271
272 def test_controller_against_foreign_plant():
273     """Swap-in proof: the controller runs, stays inside the ramp limit and
274     produces finite, in-range moves on a plant it was never tuned for."""
275     plant = _ForeignPlant()

```

```

276 check(isinstance(plant, WellSimulatorProtocol),
277 "the foreign plant does not satisfy the Protocol")
278 c = _fresh_controller()
279 Q, WHP, FLP, BHP = plant.reset()
280 u = 0.0
281 for _ in range(80):
282     prev = u
283     u = c.step({"Q": Q, "WHP": WHP, "FLP": FLP, "BHP": BHP}, u, 120.0).u_selected
284     check(abs(u - prev) <= CONTROLLER.max_move + 1e-9, "ramp limit breached on foreign plant")
285     check(0.0 <= u <= 100.0, "choke out of range on foreign plant")
286     Q, WHP, FLP, BHP = plant.step(u)
287     check(all(np.isfinite(v) for v in (Q, WHP, FLP, BHP)), "foreign plant produced NaN")
288
289
290 # ----- scenario logs
291 def test_scenario_logs_respect_every_limit():
292     import pandas as pd
293     for key in ("A", "B", "C"):
294         path = DATA_DIR / f"scenario_{key}_log.csv"
295         if not path.exists():
296             continue
297         df = pd.read_csv(path)
298         check(not df.isna().any().any(), f"NaN in scenario {key} log")
299         check(df["WHP"].between(LIMITS.whp_min, LIMITS.whp_max).all(), f"WHP breach in {key}")
300         check(df["FLP"].between(LIMITS.flp_min, LIMITS.flp_max).all(), f"FLP breach in {key}")
301         check(df["BHP"].between(LIMITS.bhp_min, LIMITS.bhp_max).all(), f"BHP breach in {key}")
302         check((df["choke_pct"].diff().abs().dropna() <= CONTROLLER.max_move + 1e-6).all(),
303               f"ramp breach in {key}")
304
305
306 def main():
307     tests = [v for k, v in sorted(globals().items()) if k.startswith("test_") and callable(v)]
308     print(f"Running {len(tests)} tests\n" + "-" * 58)
309     passed = 0
310     for t in tests:
311         try:
312             t()
313             print(f" PASS {t.__name__}")
314             passed += 1
315         except Exception as e:
316             FAILURES.append((t.__name__, e))
317             print(f" FAIL {t.__name__}: {e}")
318             if not isinstance(e, AssertionError):
319                 traceback.print_exc()
320             print("-" * 58)
321     print(f"{passed}/{len(tests)} passed")
322     if FAILURES:
323         sys.exit(1)
324
325
326 if __name__ == "__main__":
327     main()
328

```